

AOIA MAHT v2.0

Multi-Agent Hypothesis Testing Framework

Executive Summary

This document records the first major audit divergence encountered during AOIA-Core development. Multiple AI auditors reviewed the same engineering problem and produced conflicting recommendations. Instead of selecting a winner by authority or majority vote, the team resolved the dispute through experimental validation.

Project Context

AOIA-Core is a local-first inspection and audit framework. During reviewer-packaging preparation, a dispute emerged concerning CI configuration, Python import layout, and packaging strategy.

Origin Story of MAHT

During active development, the project founder proposed gathering multiple independent AI audits and resolving disagreements through testing rather than opinion. Only later were similarities identified with N-Version Programming, the Delphi Method, and Scientific Hypothesis Testing.

The Packaging Dispute

Option A proposed using `PYTHONPATH=runtime` in CI. Option B proposed packaging-oriented solutions without `PYTHONPATH`. The disagreement became the first documented AOIA audit divergence.

Perplexity Audit

Perplexity supported Option A as the safest reviewer-packaging solution. It emphasized transparency, low risk, and explicit documentation of future import-layout normalization.

Sonnet Audit

Sonnet characterized the problem as a CI configuration gap rather than a runtime defect. It recommended aligning CI with the documented validation workflow.

DeepSeek Audit

DeepSeek strongly supported Option A and emphasized that packaging work should remain separate from architectural refactoring.

Gemini / Sapphire Audit

Gemini focused on repository clarity, reviewer cognitive load, documentation structure, and long-term architecture. It highlighted broader repository concerns beyond the CI dispute.

Meta AI Red-Team Audit

Meta AI rejected Option A and argued that packaging should be corrected through a more standard Python structure. Meta emphasized reviewer perception and packaging hygiene.

Grok Red-Team Audit

Grok rejected Option A as technical debt and argued for proper packaging and editable installs. Grok focused on long-term maintainability and reviewer expectations.

Consensus Matrix

Perplexity, Sonnet, and DeepSeek converged on Option A. Meta AI and Grok converged on Option B. Gemini highlighted broader architectural concerns rather than directly resolving the dispute.

Experimental Validation

The team extracted falsifiable predictions from the competing recommendations. Option B predicted that alternative discovery approaches would work without PYTHONPATH. Option A predicted that the documented runtime import configuration would execute the test suite successfully.

Experimental Results

Meta-style discovery failed with ImportError before test execution. Option A executed successfully. Result: 470 tests passed and 4 tests were skipped.

Decision Record

The final decision selected Option A for the reviewer-packaging phase. Import-layout normalization was deferred to a later audited phase.

Methodological Analysis

The decision was based on evidence rather than authority. AI systems were treated as hypothesis generators rather than sources of truth.

Comparison to Existing Methods

The process resembles N-Version Programming, the Delphi Method, and Scientific Hypothesis Testing. AOIA does not claim to invent these methods, but adapts similar principles to multi-model AI auditing.

MAHT Framework v0.1

Step 1: Gather independent audits. Step 2: Extract competing hypotheses. Step 3: Convert hypotheses into testable predictions. Step 4: Execute experiments. Step 5: Record failures and successes. Step 6: Make evidence-based decisions.

Strengths

Avoided majority-vote thinking, reduced automation bias, created a permanent record of disagreements, and linked architectural decisions to reproducible evidence.

Weaknesses

Some skipped tests remain unexplained. Failure analysis for incorrect model recommendations should be expanded in future iterations.

Governance Proposal

Future AOIA work should trigger multi-model audits whenever uncertainty exceeds an agreed threshold. Architectural disputes should be documented and resolved through experiment whenever possible.

Long-Term Value

Preserving disagreements creates a calibration dataset for future work. Over time, the project can identify model strengths, weaknesses, and recurring error patterns.

Lessons Learned

Do not trust a single model. Do not trust a majority of models. Trust experimentally verified outcomes.

Appendix A

Summary of auditor positions and divergence patterns.

Appendix B

Summary of experimental outcomes and validation logic.

Appendix C

Future Import Layout Normalization phase reserved for later audited work.